ACADEMIA JAVA BACKEND — SPRING BOOT 4.1.x

Atividade Prática • Aula 1: REST Básico, Annotations e Coleções em Memória
Projeto Transversal: MyMovies API

1. Lembrete Teórico Conciso (Conceitos da Aula 1)

Antes de iniciar a implementação do desafio, revise os conceitos fundamentais que suportam a criação de endpoints HTTP no Spring Boot:

- **@RestController:** Define que a classe Java é um componente gerenciado pelo Spring MVC para receber requisições web e serializar o retorno dos métodos diretamente no corpo da resposta HTTP (JSON).
- **@RequestMapping("/api/movies"):** Define o prefixo base da rota da API para o recurso de domínio (ex: /api/movies).
- **@GetMapping:** Mapeia requisições do verbo HTTP GET para o método Java anotado.
- **@PathVariable:** Extrai variáveis informadas diretamente no caminho do URI (ex: /api/movies/{id}). É a forma padrão do REST para identificar um recurso individualizado.
- **@RequestParam:** Captura parâmetros de consulta (query parameters) enviados após a interrogação na URL (ex: /api/movies/search?title=matrix). Ideal para filtros e pesquisas opcionais.
- **Java 21 Record:** Estrutura imutável introduzida no Java moderno para representar dados sem boilerplate de getters, setters, equals e hashCode (ex: public record Movie(Long id, String title, String director, Integer year) {}).
- **Stream API:** Utilização da Stream API do Java (movies.stream().filter(...).toList()) para filtrar coleções em memória de forma funcional e declarativa.
- **Serialização JSON:** A biblioteca Jackson (integrada ao spring-boot-starter-web) converte automaticamente objetos e records Java retornados pelos controladores em JSON entregue ao cliente.

2. Enunciado da Atividade Prática (Passo a Passo)

O objetivo deste exercício é consolidar a criação de rotas REST de leitura para o catálogo da aplicação MyMovies. Você criará um controlador capaz de listar filmes, buscar por ID único e realizar buscas parciais por título.

Requisitos Funcionais da API

Sua API deve expor exatamente as três capacidades de consulta descritas na tabela abaixo:

Método	Endpoint / Rota	Parâmetro	Descrição / Comportamento Esperado

GET	`/api/movies`	Nenhum	Retorna a lista completa com todos os filmes em memória.
GET	`/api/movies/{id}`	id (Path Variable)	Busca o filme correspondente ao ID. Retorna o objeto Movie ou null se não encontrar.
GET	`/api/movies/search`	title (Query Param)	Retorna uma lista contendo os filmes cujo título contenha o texto informado (busca case-insensitive).

Roteiro Guiado de Execução

- **Passo 1 — Modelo:** No pacote com.academia.mymovies.model, crie o record Movie com os atributos id (Long), title (String), director (String) e year (Integer).
- **Passo 2 — Controlador:** No pacote com.academia.mymovies.controller, crie a classe MovieController anotada com @RestController e @RequestMapping("/api/movies"). Inicialize em memória uma lista fixa contendo filmes de teste.
- **Passo 3 — Endpoint Listar:** Crie o método listAll() anotado com @GetMapping que retorna List<Movie>.
- **Passo 4 — Endpoint Por ID:** Crie o método getById(@PathVariable Long id) anotado com @GetMapping("/{id}") que utiliza Stream para localizar o filme pelo ID.
- **Passo 5 — Endpoint Busca por Título:** Crie o método searchByTitle(@RequestParam(required = false) String title) anotado com @GetMapping("/search"). Utilize Stream e .toLowerCase().contains(...) para permitir pesquisas flexíveis.
- **Passo 6 — Validação:** Execute a classe MyMoviesApplication no IntelliJ, verifique se o Tomcat subiu na porta 8080 e valide os testes pelo navegador web ou via coleção no cliente HTTP Bruno.

> **DICA DE IMPLEMENTAÇÃO:** Garantir que a busca por título seja insensível a maiúsculas/minúsculas (.toLowerCase()) e que retorne todos os filmes caso o parâmetro title não seja informado.

Critérios de Aceite (Definition of Done)

- Aplicação inicializa perfeitamente no Tomcat embutido na porta 8080 sem exceções no console.
- Requisição GET /api/movies retorna Status 200 OK e a lista JSON completa de filmes.
- Requisição GET /api/movies/1 retorna o filme 'Interstellar'.
- Requisição GET /api/movies/search?title=matrix retorna apenas os filmes que contêm 'matrix' no título.
- O aluno consegue explicar o papel do @RestController, como o parâmetro da URL chega ao método Java e como o Spring converte o objeto em JSON.

3. Arquivo de Apoio

Abaixo encontra-se a massa fixa de 21 filmes para testes.

Ficheiro 1: Lista de Filmes para MovieController.java

```
…
// Lista fixa em memória com 22 filmes para testes e validação
    private final List<Movie> movies = List.of(
        new Movie(1L, "Interstellar", "Christopher Nolan", 2014),
        new Movie(2L, "The Matrix", "Lana & Lilly Wachowski", 1999),
        new Movie(3L, "Inception", "Christopher Nolan", 2010),
        new Movie(4L, "The Dark Knight", "Christopher Nolan", 2008),
        new Movie(5L, "Pulp Fiction", "Quentin Tarantino", 1994),
        new Movie(6L, "The Godfather", "Francis Ford Coppola", 1972),
        new Movie(7L, "Fight Club", "David Fincher", 1999),
        new Movie(8L, "Forrest Gump", "Robert Zemeckis", 1994),
        new Movie(9L, "The Lord of the Rings: The Fellowship of the Ring", "Peter
Jackson", 2001),
        new Movie(10L, "Star Wars: Episode V - The Empire Strikes Back", "Irvin
Kershner", 1980),
        new Movie(11L, "The Matrix Reloaded", "Lana & Lilly Wachowski", 2003),
        new Movie(12L, "The Matrix Revolutions", "Lana & Lilly Wachowski", 2003),
        new Movie(13L, "Gladiator", "Ridley Scott", 2000),
        new Movie(14L, "Back to the Future", "Robert Zemeckis", 1985),
        new Movie(15L, "Jurassic Park", "Steven Spielberg", 1993),
        new Movie(16L, "Alien", "Ridley Scott", 1979),
        new Movie(17L, "Blade Runner 2049", "Denis Villeneuve", 2017),
        new Movie(18L, "Dune", "Denis Villeneuve", 2021),
        new Movie(19L, "Dune: Part Two", "Denis Villeneuve", 2024),
        new Movie(20L, "Oppenheimer", "Christopher Nolan", 2023),
        new Movie(21L, "Avatar", "James Cameron", 2009),
    );
```